# RankResume

Project Idea — Multi-Resume vs. Job Description Matching, Ranking, and Candidate Insights(Only for IT field)

---

## 1. Problem Statement

Recruiters and hiring teams typically screen large volumes of resumes against a single job description manually, which is slow, inconsistent, and prone to overlooking well-qualified candidates. RankResume addresses this by combining resume parsing, NLP-based matching, and predictive scoring to give a recruiter a ranked, percentage-based view of how multiple candidates fit a single job description, along with a transparent breakdown of what drove each candidate's match score.

## 2. Phase 1 — Multi-Resume, Single-JD Matching, Ranking, and Candidate Insights

Goal: take the recruiter from signup through to viewing all uploaded candidates sorted into Weak, Medium, and Strong match buckets against one job description, and drilling into any candidate for a detailed score breakdown.

### 2.1 Step-by-Step Flow

**Step 1: Signup** — Recruiter submits company name, company email, and password.

**Step 2: Job Description Submission** — Recruiter submits a single job description (pasted text or uploaded file) for the role being hired for.

**Step 3: Resume Upload (multiple)** — Recruiter uploads multiple candidate resumes (PDF/DOCX) against that job description. Each resume file is stored, with a reference path saved in the database.

**Step 4: Resume Parsing** — Triggered right after each upload.

- Extract raw text: pdfplumber / PyMuPDF for PDF, python-docx / docx2txt for DOCX.
- Clean text: remove headers/footers, normalize whitespace.
- NLP extraction of skills, education, experience years, and job titles (spaCy).
- Output: a structured candidate profile in JSON form, stored separately from the raw resume file, linked to the job description it was submitted against.

**Step 5: JD Parsing** — The job description is parsed into a structured profile of required skills, experience, education, and responsibilities.

**Step 6: Login** — Standard authentication, validating credentials against Phase 1 signup data.

**Step 7: Matching Engine** — Runs after each upload or after all resumes for a JD are

uploaded.

- Each candidate profile and the single job requirement profile are vectorized (TF-IDF or free local sentence-embedding models).
- Cosine similarity is computed per candidate and converted into a match percentage (0–100%).
- Sub-scores (skills match %, experience match %, education match %) are optionally weighted into a combined score.

**Step 8: Bucketing** — Thresholds applied per candidate: below 60% → Weak; 61–90% → Medium; above 91% → Strong. Thresholds may also be tuned using k-means clustering instead of fixed cut-offs.

**Step 9: Display** — Weak / Medium / Strong sections shown on the dashboard for the given job description, sorted in descending order of match percentage within each bucket.

## 3. Phase 2 — Candidate Testing

The rough idea: After reviewing the ranked candidates, the recruiter can choose a set of shortlisted applicants for an online assessment. The system automatically generates a unique assessment link for each selected candidate and sends it to the email address extracted from their resume. The assessment contains role-specific technical questions generated from both the job description and the candidate's resume, with all responses submitted through a typing interface. Once the test is completed, the system evaluates the answers using an LLM, analyzes basic typing behaviour (such as typing speed, pauses, and editing patterns), and combines the assessment score with the resume matching score to produce a final ranked list for the recruiter. The platform is intended to support the shortlisting process before interviews, while leaving the final hiring decision to the company. **This phase is not fully scoped yet and will be detailed once the approach is finalized.**